

Bài tập về nhà

Môn Kiến trúc hướng dịch vụ

Tuần 1:

Bài 1: Tìm 3 API public thật viết về một domain quan tâm.

- Spotify Web API: là RESTful API cho phép ứng dụng truy cập metadata từ Spotify Data Catalogue, gồm nghệ sĩ, album, track, playlist, và thông tin thư viện người dùng. API chủ yếu trả về dữ liệu JSON và dùng các HTTP method chuẩn như GET, POST, PUT, DELETE
 - API type: REST
 - BASE URL: <https://api.spotify.com/>
 - Versioning: v1 được đặt trong path url <https://api.spotify.com/v1>
 - Authentication method: API sử dụng OAuth 2.0 và gửi access token trong HTTP header:
 - Authorization: Bearer ACCESS_TOKEN
 - Ví dụ request:

```
curl "https://api.spotify.com/v1/tracks/TRACK_ID" \
-H "Authorization: Bearer ACCESS_TOKEN"
```
 - Spotify dùng OAuth 2.0 cho cấp quyền truy cập tài nguyên, đặc biệt khi ứng dụng cần đọc dữ liệu private như profile hoặc playlist của user.
 - Resource identifiers: Spotify dùng ID dạng chuỗi Base-62, không phải số và cũng không phải UUID.
 - Ví dụ: id - **6rqhFgbbKwnb9MLmUQDhG6**
<https://api.spotify.com/v1/tracks/6rqhFgbbKwnb9MLmUQDhG6>
- SoundCloud API: là REST API cho phép ứng dụng làm việc với track, playlist, user, upload audio, streaming, tìm kiếm, comment, follow và like. Đây là API phù hợp nếu ứng dụng quan tâm đến nhạc do nghệ sĩ độc lập đăng tải, nội dung cộng đồng và các tính năng mạng xã hội âm nhạc.
 - API type: REST
 - BASE URL: <https://api.soundcloud.com>
 - Versioning: không có trong api công khai
 - Authentication method: SoundCloud sử dụng OAuth 2.1 with PKCE. Tất cả API request gửi access token trong header:
 - Authorization: OAuth ACCESS_TOKEN
 - Ví dụ lấy profile của người dùng đã xác thực:

```
curl "https://api.soundcloud.com/me" \
-H "accept: application/json; charset=utf-8" \
```

-H "Authorization: OAuth ACCESS_TOKEN"

- Access token có thời hạn khoảng 1 giờ. Refresh token là loại dùng một lần: sau khi refresh thành công, ứng dụng cần lưu refresh token mới được trả về.
 - Resource identifiers: dùng số
- MusicBrainz API: là REST API truy cập MusicBrainz Database — cơ sở dữ liệu metadata âm nhạc mở. API thích hợp cho media player, công cụ gắn tag nhạc, app tra cứu artist/album/song và các hệ thống quản lý metadata.
 - API type: REST
 - BASE URL: <https://musicbrainz.org/ws/2>
 - Versioning: /ws/2
 - Authentication method: Các request chỉ đọc dữ liệu công khai không cần API key và không cần OAuth. OAuth 2.0 chỉ khi ghi dữ liệu. Tuy nhiên, mọi ứng dụng bắt buộc phải gửi header User-Agent có ý nghĩa:
 - User-Agent: MyMusicProject/1.0 (contact@example.com)
 - Resource identifiers: sử dụng MBID là UUID, gồm 36 ký tự
 - Ví dụ: 65f4f0c5-ef9e-490c-ae3-909e7ae6b2ab

Bài 2: Đọc Higginbotham Ch.1 & Geewax Ch.1.

Tập trung vào: định nghĩa API, resource, và 5 đặc điểm của một API tốt. So sánh với 5 đặc điểm đã học trong slide 05 — có điểm nào Geewax/Higginbotham bổ sung?

- Theo Geewax, API là một loại giao diện (interface) đặc biệt, quy định cách hai hệ thống máy tính tương tác với nhau. API có thể tồn tại dưới nhiều hình thức như thư viện mã nguồn, package hoặc Web API. Trong đó, Web API là API được công khai qua mạng để nhiều hệ thống hoặc lập trình viên có thể gọi từ xa. API giúp hệ thống bên sử dụng gọi chức năng của hệ thống bên cung cấp mà không cần biết chi tiết cách hệ thống đó được xây dựng hay vận hành.
- Theo Higginbotham, Web API không chỉ là cơ chế kỹ thuật để kết nối các hệ thống mà còn là cách tổ chức đưa các năng lực kinh doanh (business capabilities) ra bên ngoài dưới dạng năng lực số (digital capabilities). API kết nối tổ chức với khách hàng, đối tác và nhân viên thông qua các tiêu chuẩn Web. Vì vậy, một API hiệu quả phải kết hợp ba yếu tố: năng lực kinh doanh (business capabilities), tư duy sản phẩm (product thinking) và trải nghiệm lập trình viên (developer experience).
- So với năm đặc điểm cốt lõi của một API hiện đại trong slide 05: Có hợp đồng rõ ràng, Ấn cài đặt, Có thể tái sử dụng, Độc lập với ngôn ngữ & nền tảng, Có thể versioning & quan sát được. Geewax bổ sung rằng API tốt phải hoạt động đúng yêu cầu (operational), cho phép client diễn đạt rõ ý định (expressive), đơn giản nhưng không đơn giản hóa quá mức (simple) và nhất quán, dễ dự đoán

(predictable). Higginbotham bổ sung quy trình và tư duy thiết kế API: thiết kế công tác, tập trung vào kết quả để đem lại giá trị, lựa chọn phong cách thiết kế API phù hợp với nhu cầu, ưu tiên tài liệu và lập kế hoạch phù hợp để API tồn tại lâu dài.

Bài 3: Hoàn thiện bài 6 + mở rộng.

Sau khi CRUD chạy được, thêm: (a) tìm kiếm GET /books?q=...; (b) sort theo ?sort=title; (c) bắt buộc field year là số ≥ 1900 .

Dưới đây là những phần chỉnh sửa mở rộng bài 6:

- Thêm query parameter, sort theo title:

```
@app.route("/books", methods=["GET"])
def list_books():
    n = int(request.args.get("limit", 100))
    query = request.args.get("q", "").strip().lower()
    sort_by = request.args.get("sort", "").strip().lower()
    books = BOOKS
    if query:
        books = [
            book
            for book in books
            if query in book["title"].lower()
            or query in book["author"].lower()
        ]
    if sort_by == "title":
        books = sorted(
            books,
            key=lambda book: book["title"].lower(),
        )
    books = books[:n]
    return jsonify(books), 200
```

- Thêm điều kiện năm ≥ 1900

```
@app.route("/books", methods=["POST"])
def create_book():
    global _next
    body = request.get_json(silent=True) or {}
    t, a, y = body.get("title"), body.get("author"),
body.get("year")
    if not t or not a or not y:
        return {"error": "need title + author + year"}, 400
    if not isinstance(y, int) or y < 1900:
        return {"error": "year must be an integer >= 1900"}, 400
    book = {"id": _next, "title": t, "author": a, "year": y}
```

```
_next += 1; BOOKS.append(book)
return jsonify(book), 201,
{"Location":f"/books/{book['id']}"} }
```